BÀI 9
ĐỒ HỌA

MỤC TIÊU

Người học sau khi học xong bài 9 sẽ có các khai niệm cơ bản về các vấn đề sau:

- Khái niệm đồ họa
- Khởi động hệ đồ họa
- Lỗi đồ họa
- Chọn kiểu đường
- Tô điểm và tô miền

KIẾN THỨC CẦN CÓ

Các kiến thức cần thiết:

- Học xong bài 8;
- Có kiến thức về đường, điểm, giao diện;
- Phân tích hình ảnh, đồ họa;
- Thực hiện các bài toán về điểm, diện tích, hình học…;
- Khuyến nghị học môn Tin Học Đại Cương.

NỘI DUNG

- Khái niệm đồ họa
- Khởi động hệ đồ họa
- Lỗi đồ họa
- Mầu và mẫu
- Vẽ và tô màu
- Chọn kiểu đường
- Cửa sổ (Viewport)
- Tô điểm, tô miền
- Xử lý văn bản trên màn hình đồ họa
- Cắt hình, dán hình và tạo ảnh chuyển động
- Một số chương trình đồ họa
- Đồ họa 256 màu trong C

9.1. KHÁI NIỆM ĐỒ HỌA

- Màn hình hiển thị chia làm 25 dòng, với 40 hoặc 80 cột.
- Mỗi tọa độ đều có thể được xử lý và coi như là 1 điểm ảnh (pixel)
- Số lượng điểm anh có trong 1 màn hình được gọi là độ phân giải
- Để có thể thực hiện được đồ họa, C dùng các tập tin điều khiển và các font chữ.

GOTH.CHR
LITT.CHR
SANS.CHR
TRIP.CHR

ATT.BGI
CGA,BGI
EGA,VGA,BGI.
HERC.BGI
IBM8514.BGI
PC3270.BGI

1. KHÁI NIỆM ĐỒ HỌA (tiếp theo)

- Tùy từng chế độ hiển thị, số điểm ảnh dọc và ngang cũng khác nhau.
- Chế độ CGA
  - x = 320
  - y = 200
- Chế độ VGA
  - x = 639
  - y = 479
- Có thể dùng hàm để xác định tọa độ x,y lớn nhất getmaxx, getmaxy.



2. KHỞI ĐỘNG HỆ THỐNG ĐỒ HỌA

Khởi động hệ đồ họa là xác định màn hình sẽ sử dụng trong chương trình, lựa chọn mode đồ họa được sử dụng theo cấu trúc sau:

- Cấu trúc:

```
void initgraph(int *graphdriver, int *graphmode, char *driverpath);
```

- graphdriver : loại đồ họa được sử dụng
- graphmode: chế độ đồ họa, cho phép lựa chọn các chế độ đồ họa khác nhau với độ phân phải từ 320x200 đến 1024x768
- driverpath: đường dẫn chứa tập tin điều khiển đồ họa

Ví Dụ:

```
#include "graphic.h"
void main()
{
int mh = EGA, mode = EGALO;
initgraph(&mh, &mode,"c:\tc");
- - - - .
}
```

3. LỖI ĐỒ HỌA

- Khi khởi động hệ thống đồ họa, nếu không có chương trình điều khiển đồ họa thì sẽ có lỗi xảy ra.
- Lỗi đồ họa thường phát sinh vùng các hàm.
- Để biết được lỗi loại gì thì có thể dùng hàm graphresult.
- Để hiển thị loại lỗi có thể dùng hàm grapherrormsg(maloi)

Cú pháp:

```
int maloi;
maloi = graphresult();
printf("\n Loi do hoa la: %s", grapherrormsg(maloi));
```

3. LỖI ĐỒ HỌA (tiếp theo)

Một số lỗi đồ họa cơ bản kèm theo giá trị tương ứng:

Hằng	Trị
grOK	0
grNoInitGraph	-1
grNotDetect	-2
grFileNotFound	-3
grInvalidDriver	-4
grNoLoadMem	-5
grNoScanMem	-6
grNoFloodMem	-7

Hằng	Trị
grFontNoFound	-8
grNoFontMem	-9
grInvalidMode	-10
grError	-11
grIOerror	-12
grInvalidFont	-13
grInvalidFontNum	-14

4. MẦU VÀ MẪU

- Hàm thiết lập màu nền:

```
void setbkcolor(int color);
```

- Hàm thiết lập màu đường vẽ:

```
void setcolor(int color);
```

- Hàm thiết lập mẫu tô màu:

```
void setfillstyle(int color);
```

- Hàm chọn giải màu:

```
void setpalette(int
colornum,int color);
```

TêN Hằng	Giá trị
BLACK	0
BLUE	1
GREEN	2
CYAN	3
RED	4
MAGENTA	5
BROWN	6
LIGHTGRAY	7
DARKGRAY	8
LIGHTBLUE	9
LIGHTGREEN	10
LIGHTCYAN	11
LIGHTRED	12
LIGHTMAGENTA	13
YELLOW	14
WHITE	15

4. MẦU VÀ MẪU (tiếp theo)

- Hàm xác định giải mầu hiện hành

Cú pháp:

```
void getpalatte (struct palattetype *platte);
```

➢ palattetype là kiểu được định nghĩa trước như sau

Cú pháp:

```
#define MAXCOLOR 15
struct palattetype
{
unsigned char size;
unsigned char color[MAXCOLOR +1];
};
```

- Hàm getcolor: Trả về màu đã xác định trước đó bằng hàm setcolor
- Hàm getbkcolor: Trả về màu đã xác định trước đó bằng hàm setbkcolor
- Hàm getmaxcolor : Trả về số lượng màu cực đại của giải màu hiện hành. Trên 256k EGA, hàm getmaxcolor trả về giá trị 15

5. VẼ VÀ TÔ MÀU

Quá trình vẽ và tô màu có thể được chia thành 4 nhóm chính như sau:

- Đường tròn và hình tròn
- Đường gấp khúc, hình đa giác
- Đường thẳng
- Hình chữ nhật

5. VẼ VÀ TÔ MÀU (tiếp theo)

Nhóm đường tròn và hình tròn:

- Cung tròn:

Cú pháp:

```
void arc(int x,int y, int gd, int gc, int r);
```

- x,y: tọa độ trung tâm
- gd: góc đầu
- gc: góc cuối
- r: bán kính
- arc: tên hàm vẽ cung tròn
- void: Hàm tự thực hiện và ko có giá trị trả về

Chú ý: Các gd,gc nằm trong khoảng 0 đến 360

5. VẼ VÀ TÔ MÀU (tiếp theo)

Nhóm đường tròn và hình tròn (tiếp):

- Đường tròn

Cú pháp:

```
void circle(int x,int y, int r);
```

- x,y: tọa độ trung tâm
- r: bán kính
- circle: tên hàm vẽ đường tròn
- void: Hàm tự thực hiện và ko có giá trị trả về

Chú ý: nếu gd = 0 và gc = 360 thì hàm vẽ đường tròn sẽ trùng dạng với hàm vẽ cung tròn

5. VẼ VÀ TÔ MÀU (tiếp theo)

Nhóm đường tròn và hình tròn (tiếp):

- Đường ellipse:

Cú pháp:

```
void ellipse(int x,int y, int gd, int gc, int xr, int  yr);
```

  - xr: bán kính trục x
  - yr: bán kính trục y
  - Các tham số còn lại tương đương như của hình tròn

- Hình quạt:

Cú pháp:

```
void pieslice(int x,int y, int gd, int gc, int r);
```

  - pieslice : tên hàm
  - x,y: tọa độ tâm
  - gd,gc: góc đầu và góc cuối
  - r: bán kính

5. VẼ VÀ TÔ MÀU (tiếp theo)

Đường gấp khúc và đa giác:

- Để vẽ 1 đường gấp khúc qua n điểm, ta làm theo các bước sau:
  - Đưa tọa độ các điểm vào 1 mảng a kiểu int
  - Viết lời gọi hàm:

```
drawnpoly(n,a);
```

    - drawnpoly : tên hàm
    - n: số điểm cần vẽ
    - a: mảng kiểu int chứa n điểm
    - nếu điểm đầu và cuối trung nhau ta sẽ có 1 hình đa giác
  - Ví dụ về hình gấp khúc và 2 hình tam giác

5. VẼ VÀ TÔ MÀU (tiếp theo)

Đường thẳng:

- Vẽ đường thẳng giữa 2 điểm x1,y1 tới x2,y2

```
void line (int x1,int y1, int x2, int y2);
```

- Vẽ đường thẳng từ con trỏ hiện tại tới vị trí x,y

```
void lineto (int x,int y);
```

- Vẽ con trỏ dịch chuyển 1 đoạn dx,dy so với vị trí hiện tại

```
void linerel (int dx,int dy);
```

- Di chuyển con trỏ tới vị trí mới x,y

```
void moveto(int x,int y);
```

5. VẼ VÀ TÔ MÀU (tiếp theo)

- Hình chữ nhật:

```
void rectangle(int x1,int y1,int x2,int y2);
```

  - rectangle : lời gọi hàm
  - x1,y1: đỉnh trên bên trái
  - x2,y2: đỉnh dưới bên phải
- Hình chữ nhật được tô màu:

```
void bar(int x1,int y1,int x2,int y2);
```

  - bar : lời gọi hàm tô màu
  - x1,y1: đỉnh trên bên trái
  - x2,y2: đỉnh dưới bên phải
- Xem thêm ví dụ minh họa về vẽ hình chữ nhật

6. CHỌN KIỂU ĐƯỜNG

Dạng đường được xác định bởi tham số linestyle, tham số này có thể tác động lên các thủ tục vẽ đường đã trình bày ở phần trước như line, lineto, rectangle…

- Các giá trị khả dĩ của kiểu đường thể hiện ở bảng bên
- Bề dày của đường được mô tả bằng tham số thickness

SOLID_LINE	0
DOTTED_LINE	1
CENTER_LINE	2
DASHE_LINE	3
USERBIT_LINE	4

NORM_WIDTH =1
THICK_WIDTH = 3

6. CHỌN KIỂU ĐƯỜNG (tiếp theo)

Để nhận giá trị hiện thời của dạng đường và bề dày đường, ta dùng cú pháp sau:

Cú pháp:

```
void getlinesetting(struct linesettingstype *lineinfo);
```

- linesettingstype: cấu trúc được định nghĩa như sau:

```
struct linesettingstype
{
int linestyle;
unsigned int upattern;
int thickness;
};
```

7. CỬA SỔ (Viewport)

- Cửa sổ là một vùng chữ nhật trên màn hình đồ họa như màn hình windows trong textmode.
- Cửa sổ được thiết lập như sau:

```
void setviewport(int x1,int y1, int x2, int y2, int clip)
```

  - setviewport : thiết lập cửa sổ
  - x1,y1 : tọa độ góc trên bên trái
  - x2,y2 : tọa độ góc dưới bên phải
  - clip = 1: cho phép vẽ ra ngoài cửa sổ
  - clip = 0: không cho phép vẽ ra ngoài cửa sổ

7. CỬA SỔ (Viewport) (tiếp theo)

- Hàm xác định cửa sổ hiện hành:

```
void getviewsetting(struct viewporttype *vp);

struct viewporttype
{
int left, top, right, bottom;
int clip;
};
```

- Hàm xóa cửa sổ:

```
void clearviewport(void);
```

- Hàm xóa màn hình:

```
void cleardevice(void);
```

7. CỬA SỔ (Viewport)(tiếp theo)

- Nhờ sử dụng cửa sổ nên ta có thể viết các chương trình đồ họa theo tọa độ âm dương, với gốc 0 chính là vị trí con trỏ nằm giữa màn hình:

```
int xc,yc;
xc = getmaxx()/2; yc = getmaxy()/2;
setviewport(xc,yc,getmaxx(),getmaxy(),0)
```

- Quá trình khai báo này sẽ chia cửa sổ ra làm 4 phần khác nhau:
  - góc trái trên: x âm, y âm
  - góc trái dưới: x âm, y dương
  - góc phải trên: x dương , y âm
  - góc phải dưới: x dương, y dương

8. TÔ ĐIỂM, TÔ MIỀN

- Hàm tổ điểm với màu được xác định trước bởi hàm setcolor:

```
void putpixel(int x, int y, int color);
```

- Hàm lấy thông tin điểm ảnh x,y:

```
unsigned getpixel(int x, int y);
```

- Hàm tô cho một miền trên màn hình:

```
void foodfill(int x, int y, int border);
```

  - foodfill : tên hàm
  - x,y: tọa độ điểm nằm trong miền cần tô
  - border: Thông tin mã màu
  - Xét ví dụ vẽ đường tròn đỏ trên màn hình xanh

9. XỬ LÝ VĂN BẢN TRÊN MÀN HÌNH ĐỒ HỌA

Hiển thị văn bản trên màn hình đồ họa thông qua 2 cách sau:

- Cách 1:

```
void outtext(char *s);
```

  - Dùng con trỏ s trỏ tới dữ liệu
  - Đưa dữ liệu này ra hiển thị

- Cách 2:

```
void outtextxy(int x, int y, char *s);
```

  - Dùng con trỏ s trỏ tới dữ liệu
  - Đưa dữ liệu này ra hiển thị tại vị trí x,y

Chú ý: trong chế độ đồ họa, ta vẫn có thể sử dụng các hàm scanf, getch()…

9. XỬ LÝ VĂN BẢN TRÊN MÀN HÌNH ĐỒ HỌA (tiếp theo)

Khi cần trình bày văn bản với các kiểu chữ khác nhau, ta dùng các font có đuôi .CHR với cú pháp sau:

```
void settextstyle(int font, int direction, int charsize);
```

- Hàm chỉ tồn tại nếu có các font tương ứng
- Direction có thể nhận 1 trong 2 giá trị
  - HORIZ_DIR = 0 (hiển thị ngang, từ trái qua phải)
  - VERT_DIR = 1 (hiển thị đứng, từ dưới lên trên)
- charsize: Hệ số phóng to ký tự có giá trị 1, đến10

Ví Dụ:

```
settextstyle(3,VERT_DIR,2);
outtext(50,50,"HELLO")'
```

9. XỬ LÝ VĂN BẢN TRÊN MÀN HÌNH ĐỒ HỌA (tiếp theo)

Khi cần thay đổi vị trí hiển thị của text, có thể dùng hàm sau:

```
void settextjustify(int horiz, int vert);
```

- settextjustify : tên hàm
- horiz, vert tham số xác định vị trí của text, có thể có các giá trị sau:

Tham số	Giá trị	Ý nghĩa
LEFT_TEXT	0	Căn lề trái
CENTER_TEXT	1	Căn lề giữa
RIGHT_TEXT	2	Căn lề phải
BOTTOM_TEXT	0	Căn lề dưới
TOP_TEXT	2	Căn lề trên

9. XỬ LÝ VĂN BẢN TRÊN MÀN HÌNH ĐỒ HỌA (tiếp theo)

Muốn thay đổi bề rộng và chiều cao của văn bản, ta dùng 2 hàm textheight và textwidth:

```
void textheight(char *s);
```

- Trả về chiều cao của chuỗi do con trỏ s trỏ tới. Nếu font dùng định dạng bitmap 8x8 và khuếch đại = 1 thì textheight("H")=8;

```
void textwidth(char *g);
```

- Trả về chiều dài của chuỗi do con trỏ g trỏ tới. Dựa vào chiều dài của chuỗi, kích thước font, hệ số khuếch đại chữ, giá trị g sẽ là số pixel của chữ.

10. CẮT HÌNH, DÁN HÌNH VÀ TẠO ẢNH CHUYỂN ĐỘNG

- Muốn xác định kích thước của ảnh ta dùng hàm sau:

```
unsigned imagesize(int x1, int y1, int x2, int y2);
```

  - x1,y1: tọa độ góc trên bên trái của ảnh
  - x2,y2: tọa độ góc dưới bên phải của ảnh
- Xác định vùng nhớ mới được cấp phát thông qua hàm malloc:

```
#include <malloc.h>
void *malloc(unsigned n);
```

  - malloc: tên hàm, kiểu con trỏ
  - n: số byte mới được cấp phát

10. CẮT HÌNH, DÁN HÌNH VÀ TẠO ẢNH CHUYỂN ĐỘNG (tiếp theo)

- Hàm getimage:

```
void getimage(int x1, int y1, int x2, int y2, void * bitmap)
```

  - chép các điểm ảnh của hình chữ nhật x1,y1 -> x2,y2 và các thông tin khác tới vùng nhớ do con trỏ ảnh trỏ tới.
  - Vùng nhớ này được cho bởi hàm malloc.
  - Độ lớn của vùng nhớ này được xác định bằng hàm imagesize

- Hàm putimage:

```
void putimage(int x1, int y1, void * bitmap, int copymode)
```

  - Copy mode có thể có các trường hợp sau:
    - COPY_PUT = 0 : copy nguyên bản
    - XOR_PUT =1 : dùng phép toán xor cho điểm ảnh
    - OR_PUT = 2 : dùng phép toán or cho điểm ảnh
    - AND_PUT = 3 : dùng phép toán and cho điểm ảnh
    - NOT_PUT = 4 : đảo ảnh

10. CẮT HÌNH, DÁN HÌNH VÀ TẠO ẢNH CHUYỂN ĐỘNG (tiếp theo)

Để tạo ảnh chuyển động cho ảnh ta thực hiện theo quy tắc sau:

- Vẽ hình : vẽ lại ảnh mới hoặc đọc lại bộ nhớ
- Trễ : trễ thời gian
- Xóa hình :
  - Dùng hàm cleandriver
  - Dùng XOR_PUT để xếp chồng lên ảnh cần xóa
  - Lưu màn hình
- Vẽ hình tiếp theo
- Trễ

11. ĐỒ HỌA 256 MÀU TRONG C

Hầu hết các chương trình C sử dụng 16 màu. Muốn sử dụng 256 màu thì cần phải có thêm thư viện và hàm khởi tạo như sau:

- Cần có thư viện SVGA256.BGI cho phép hiển thị 256 màu

- Khai báo hàm khởi tạo 256 màu:

```
int mh, mode;
mh = installuserdriver("SVGA256", NULL);
mode = 16;
initgraph(&mh,&mode,"");
```

TÓM LƯỢC CUỐI BÀI

- Bài 9 quan trọng trong nội dung chương trình, nó cung cấp cho người học tổng quan về các kỹ thuật đồ họa cơ bản của C.
- Việc phân tích ảnh, đường, hình, cấu trúc màu… cần phải được thực hiện trước khi lập trình hiển thị.
- Tọa độ điểm ảnh, con trỏ chuột có vai trò quan trọng khi cần hiển thị các hiệu ứng ảnh.
- Có khá nhiều hàm tác động tới ảnh;
- Có thể thực hiện ảnh động bằng việc kết hợp ảnh tĩnh và trễ thời gian nhằm xây dựng các hiệu ứng hoạt hình.